
Kubernetes

Workflow, Manifest Reading & Interview Guide

Processes | How to read YAML | What to focus on | Real interview Q&A

What is inside

- **Part 1** - Kubernetes workflow structure: how a request flows, the reconcile loop, and the app delivery pipeline
- **Part 2** - How to read a manifest file, and what to focus on most
- **Part 3** - Every process, topic by topic: what it solves, key commands, focus points
- **Part 4** - End-to-end real-world deployment walkthrough
- **Part 5** - 50 real interview questions with short answers (including scenario questions)
- **Part 6** - Command cheat sheet and a 7-day revision plan

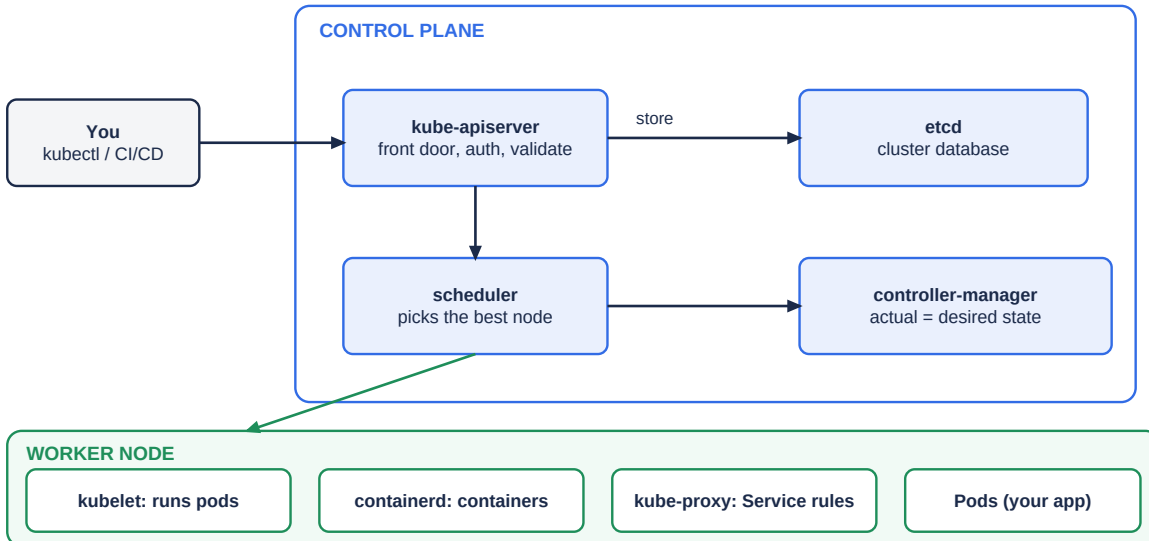
How to use this guide

Read Parts 1-2 first (they unlock everything else). Practise Part 3 commands on a local kind cluster. Before an interview, read Part 5 out loud and answer in your own words with one real example.

Part 1 - Kubernetes Workflow Structure

Kubernetes is a **desired-state system**. You declare what you want (3 replicas of image X); controllers continuously compare reality with that declaration and fix any difference. You never say *how*, only *what*.

1.1 Architecture and request flow



What happens on `kubectl apply -f deploy.yaml`:

- `kubectl` sends the YAML to the **API server**, which authenticates (who are you), authorizes with RBAC (may you), and validates/admits it (PSS, quotas).
- The object is saved in **etcd**. Nothing runs yet; only the desired state is recorded.
- The **Deployment controller** creates a **ReplicaSet**, which creates **Pod** objects (no node assigned yet).
- The **scheduler** sees unscheduled pods, filters nodes (resources, taints, affinity) and scores them, then binds each pod to a node.
- The node's **kubelet** sees a pod assigned to it, asks the container runtime to pull the image and start containers, and runs probes.
- **kube-proxy / CNI** program networking so **Services** route to the ready pods (those passing readiness).

1.2 The reconcile loop (the one idea behind everything)

observe actual state -> compare with desired state (etcd) -> act to close the gap -> repeat

Example: you asked for 3 pods, a node dies, 1 pod is lost
-> **ReplicaSet** notices `2 != 3` -> creates 1 new pod.

This is why Kubernetes 'self-heals', why **Operators** work (CRD + your own controller), and why `kubectl delete pod` on a **Deployment** pod just brings a new one back.

1.3 Object hierarchy

You create	It creates	Which creates
Deployment	ReplicaSet (one per version)	Pods
StatefulSet	Pods with stable names + PVC each	-
CronJob	Job (per schedule tick)	Pods
Service	EndpointSlices (list of ready pod IPs)	-
Ingress	Rules read by the ingress controller	-

Tip: when something is 'stuck', walk down the chain: `kubectl get deploy, rs, pods` then describe the lowest broken one.

1.4 App delivery workflow (code to production)



Real-world rule: CI builds and scans the image and pushes it with an immutable tag (git SHA); CD (kubectl, Helm or GitOps tools like Argo CD) updates the manifest and Kubernetes performs a rolling update, gated by readiness probes. Monitoring and alerts close the loop, and `kubectl rollout undo` is the escape hatch.

Part 2 - How to Read a Manifest File

Every manifest has the same 4 top-level keys. Read in this order and you can understand any YAML, even for objects you have never seen.

Key	Question it answers
apiVersion	Which API group/version? (apps/v1, v1, batch/v1, networking.k8s.io/v1)
kind	What type of object? (Deployment, Service, ...)
metadata	Who is it? name, namespace, labels, annotations
spec	What do I want? (desired state - differs per kind). <i>status</i> is filled by Kubernetes, never write it.

2.1 Annotated Deployment (read top to bottom)

```
apiVersion: apps/v1           # (1) group/version for Deployment
kind: Deployment              # (2) the object type
metadata:
  name: web                   # (3) unique name inside the namespace
  namespace: shop             # (3) WHERE it lives (forgotten namespace = 'default!')
  labels: {app: web}          # (3) tags for grouping
spec:
  replicas: 3                  # (4) desired pod count
  strategy:
    rollingUpdate: {maxSurge: 1, maxUnavailable: 0} # zero-downtime updates
  selector:
    matchLabels: {app: web}    # (5) MUST match template labels below
  template:                    # (6) the Pod blueprint (a mini Pod manifest)
    metadata:
      labels: {app: web}       # (5) Services/ReplicaSet find pods by this
    spec:
      containers:
        - name: web
          image: myrepo/web:1.4.2 # (7) pinned tag, never :latest
          ports: [{containerPort: 8080}]
          env:
            - name: DB_PASS
              valueFrom: {secretKeyRef: {name: db-secret, key: password}} # (8) config from Secret
          resources:             # (9) requests = scheduling, limits = hard cap
            requests: {cpu: 100m, memory: 128Mi}
            limits: {cpu: 500m, memory: 256Mi}
          readinessProbe:         # (10) gate traffic until healthy
            httpGet: {path: /healthz, port: 8080}
          livenessProbe:          # (10) restart if stuck
            httpGet: {path: /healthz, port: 8080}
            initialDelaySeconds: 10
          volumeMounts: [{name: data, mountPath: /data}]
      volumes:
        - name: data
          persistentVolumeClaim: {claimName: web-pvc} # (11) storage by claim
```

2.2 What to focus on most (interview + real-life priority)

Priority	Focus on	Why it matters
1	Labels and selectors (Service selector, Deployment selector, template labels)	#1 cause of 'Service has no endpoints' and 'Deployment won't create pods'
2	Ports : containerPort, Service port vs targetPort, nodePort	Wrong targetPort = connection refused with healthy pods
3	Image + tag and imagePullSecrets	ImagePullBackOff; :latest causes unpredictable rollouts
4	Resources (requests/limits)	Pending pods, OOMKilled, HPA not working without requests
5	Probes (readiness / liveness / startup)	Zero-downtime deploys vs restart loops
6	Namespace in metadata and in cross-references	Secrets/ConfigMaps/PVCs must be in the SAME namespace as the pod
7	Env / volumes : names that reference ConfigMap, Secret, PVC	CreateContainerConfigError if the name or key is wrong
8	Strategy / update policy	Controls downtime during releases
9	Security : securityContext, serviceAccountName, RBAC rules	Least privilege; PSS compliance
10	Scheduling : nodeSelector, affinity, tolerations	Pods Pending on tainted nodes

2.3 The label wiring (most important diagram)



Break any of these 3 links and traffic stops:

- 1) Ingress backend service name/port = Service name/port
- 2) Service selector = Pod labels
- 3) Service targetPort = containerPort in the Pod (check with: `kubectl get endpoints <svc>` -> must NOT be empty)

2.4 Reading shortcuts

- `kubectl explain deployment.spec.strategy --recursive` - built-in field docs, no browser needed.
- `kubectl apply --dry-run=client -o yaml -f x.yaml` or `kubectl create deploy web --image=nginx --dry-run=client -o yaml` - generate a starter manifest.
- `kubectl diff -f x.yaml` - see what would change before applying.
- `kubectl get pod web-xyz -o yaml` - read the live object including *status* (conditions, restarts, node, IP).
- YAML rules: spaces only (2 per level), - starts a list item, strings that look like numbers/booleans need quotes ("true", "80"), --- separates multiple objects in one file.

2.5 What changes per kind (the 'spec' cheat sheet)

Kind	Read these spec fields first
Pod	containers (image, command, ports, env), volumes, restartPolicy, initContainers
Deployment	replicas, selector, strategy, template
StatefulSet	serviceName (headless svc), volumeClaimTemplates, podManagementPolicy
DaemonSet	selector, template (no replicas), tolerations
Job / CronJob	completions, parallelism, backoffLimit / schedule, concurrencyPolicy, restartPolicy (Never or OnFailure only)
Service	type, selector, ports (port, targetPort, nodePort)
Ingress	ingressClassName, rules (host, path, backend), tls
PV / PVC	capacity/requests, accessModes, storageClassName, reclaimPolicy
HPA	scaleTargetRef, min/maxReplicas, metrics (target %)
Role / Binding	rules (apiGroups, resources, verbs) / subjects + roleRef
NetworkPolicy	podSelector, policyTypes, ingress/egress (from/to, ports)
CRD	group, names, scope, versions[].schema

Part 3 - Every Process, Topic by Topic

Format: **Problem solved** / **Key idea** / **Commands** / **Focus**. Examples use the repo folders 00 to 11.

3.1 Setup and kubectl

Problem solved: Running containers by hand across servers does not scale.

Key idea: Control plane + worker nodes; kind for local, kubeadm/EKS for real.

```
kind create cluster --name dev --config 00-setup/kind-config.yaml
kubectl get nodes -o wide
kubectl config get-contexts ; kubectl config use-context <ctx>
```

Focus: Know the components (apiserver, etcd, scheduler, controller-manager, kubelet, kube-proxy) and what each does.

3.2 Pods, namespaces, labels, annotations

Problem solved: Containers need a unit that shares network/storage, and a way to organize objects.

Key idea: Pod = 1+ containers, one IP. Namespace = virtual cluster. Labels select; annotations only describe.

```
kubectl apply -f 01-core-concepts/namespace-pod-labels.yaml
kubectl get pods -n nginx --show-labels -l 'app=nginx,env in (dev,qa)'
kubectl label pod nginx-pod -n nginx tier=frontend --overwrite
```

Focus: Init container (runs first, to completion) vs sidecar (runs alongside: logging, proxy). Pods are mortal; never rely on a pod IP.

3.3 Workloads

Problem solved: Apps crash, need updates, state, per-node agents and scheduled work.

Key idea: Deployment (stateless) | StatefulSet (stable name + own disk) | DaemonSet (1 per node) | Job / CronJob (batch).

```
kubectl set image deploy/web web=myrepo/web:1.5 -n shop
kubectl rollout status deploy/web -n shop
kubectl rollout undo deploy/web -n shop
kubectl scale deploy/web --replicas=5 -n shop
kubectl create job manual --from=cronjob/minute-backup -n nginx
```

Focus: Rolling update settings (maxSurge/maxUnavailable), rollback, StatefulSet headless Service + volumeClaimTemplates, CronJob concurrencyPolicy.

3.4 Networking

Problem solved: Pod IPs change; how do apps find each other and the outside world?

Key idea: Service gives a stable IP/DNS and load-balances. Ingress does HTTP routing/TLS. NetworkPolicy is the pod firewall.

```
kubectl get endpoints web -n shop          # empty = selector mismatch / pods not Ready
kubectl port-forward svc/web 8080:80 -n shop
# DNS: <svc>.<namespace>.svc.cluster.local
```

Focus: ClusterIP vs NodePort vs LoadBalancer; port vs targetPort; Ingress needs a controller; policies need a CNI that enforces them (Calico/Cilium).

3.5 Storage, ConfigMaps, Secrets

Problem solved: Container disk is lost on restart; config and passwords should not live in images.

Key idea: PVC requests storage, PV is the disk, StorageClass provisions it dynamically. ConfigMap = config, Secret = sensitive config.

```
kubectl get pv,pvc,sc -A
kubectl create secret generic db --from-literal=password='S3cr3t!'
kubectl get secret db -o jsonpath='{.data.password}' | base64 -d
```

Focus: PV is cluster-scoped (no namespace). Access modes (RWO/RWX), reclaim policy (Retain/Delete). Secrets are base64, NOT encrypted, unless etcd encryption is enabled.

3.6 Scaling and scheduling

Problem solved: Traffic changes; pods must land on the right nodes; teams must not starve each other.

Key idea: HPA (more pods), VPA (right-size pods), Cluster Autoscaler (more nodes). Affinity attracts, taints repel, tolerations allow.

```
kubectl apply -f 05-scaling-scheduling/hpa.yaml ; kubectl get hpa -w -n apache
kubectl taint node worker1 dedicated=gpu:NoSchedule
kubectl describe quota -n apache ; kubectl top pods -n apache
```

Focus: HPA needs metrics-server AND CPU requests. Limits: CPU throttles, memory OOMKills. Probes: startup / readiness / liveness. Quota = namespace cap, LimitRange = defaults.

3.7 Cluster administration

Problem solved: Who may do what? How do we upgrade safely? How do we extend the API?

Key idea: RBAC = Role + Binding. Upgrade = control plane first, then drain/upgrade/uncordon each node. CRD = new object type.

```
kubectl auth can-i delete pods -n apache --as=system:serviceaccount:apache:apache-user
kubectl drain worker1 --ignore-daemonsets --delete-emptydir-data
kubectl uncordon worker1
kubectl get crd ; kubectl get devopsbatch
```

Focus: Least privilege; one minor version at a time; back up etcd before upgrading; PodDisruptionBudgets protect availability during drains.

3.8 Monitoring and logging

Problem solved: You must know about problems before users do.

Key idea: Metrics Server (kubectl top, HPA), Prometheus + Grafana + Alertmanager for metrics, Fluent Bit + Loki/ELK for logs.

```
kubectl top nodes ; kubectl top pods -A --sort-by=memory
kubectl logs <pod> -c <ctr> --previous -f
helm install kube-prometheus-stack \
  prometheus-community/kube-prometheus-stack -n monitoring --create-namespace
```

Focus: Pod logs die with the pod, so centralize them. Alert on symptoms (errors, latency, restarts), use the four golden signals: latency, traffic, errors, saturation.

3.9 Advanced: Helm, Operators, service mesh, API

Problem solved: Hand-written YAML is repetitive; stateful apps need operational know-how; microservices need secure traffic control.

Key idea: Helm = templated packages. Operator = CRD + controller. Mesh = sidecar proxies (mTLS, canary, retries). Everything is a REST API call.

```
helm install web ./helm-chart --set replicaCount=3 ; helm upgrade web ./helm-chart ; helm rollback web 1
helm template web ./helm-chart | less
kubectl get --raw /apis | head ; kubectl get pods -v=8
```

Focus: Helm release history/rollback, values per environment. Do not add a service mesh for a handful of services.

3.10 Security

Problem solved: One bad image, open pod or leaked secret can compromise the cluster.

Key idea: Layers: image, pod, network, secrets, access, cluster.

```
kubectl label ns apps pod-security.kubernetes.io/enforce=restricted
trivy image myrepo/web:1.4.2 --severity HIGH,CRITICAL
# securityContext: runAsNonRoot, readOnlyRootFilesystem, allowPrivilegeEscalation:false, drop ALL caps
```

Focus: PSS levels (privileged < baseline < restricted), image scanning in CI, default-deny NetworkPolicy, encryption at rest for Secrets (or external secret managers).

3.11 Cloud-native Kubernetes

Problem solved: Running and upgrading the control plane yourself is costly.

Key idea: EKS / AKS / GKE manage it. Cluster Autoscaler or Karpenter add nodes. Spot nodes cut cost 60-90%.

```
eksctl create cluster -f 10-cloud-native/eksctl-cluster.yaml
aws eks update-kubeconfig --name demo-cluster
eksctl delete cluster -f 10-cloud-native/eksctl-cluster.yaml # avoid surprise bills
```

Focus: Spot is for stateless, interruption-tolerant work: use tolerations, 2+ replicas, PDBs, graceful shutdown. Always set requests so autoscaling can decide.

3.12 Debugging and troubleshooting

Problem solved: Something will break; a method beats guessing.

Key idea: get -> describe (Events) -> logs (--previous) -> events -> exec/debug.

```
kubectl get pods -o wide ; kubectl describe pod <p> ; kubectl logs <p> --previous
kubectl get events --sort-by=.lastTimestamp
kubectl debug -it <pod> --image=busybox --target=<ctr>
```

Focus: Memorize the status table in Part 5.8.

Part 4 - End-to-End Real-World Walkthrough

Scenario: deploy an online shop - **web** (stateless), **MySQL** (stateful), a nightly **backup**, public HTTPS, autoscaling and monitoring.

Step	What you do	Manifest / command
1	Create namespaces per environment	Namespace shop, ResourceQuota
2	Config and secrets	ConfigMap (settings), Secret (DB password)
3	Database	StatefulSet + headless Service + volumeClaimTemplates (PVC per pod)
4	Web app	Deployment: pinned image, requests/limits, readiness + liveness probes, rolling update
5	Expose	ClusterIP Service -> Ingress (host + TLS)
6	Protect	NetworkPolicy: only web -> mysql:3306; PSS restricted; ServiceAccount + Role for the app
7	Scale	HPA on CPU (needs metrics-server); Cluster Autoscaler for nodes
8	Batch	CronJob: nightly mysqldump to a volume/S3 with concurrencyPolicy Forbid
9	Observe	Prometheus ServiceMonitor, alert on restarts/5xx, logs via Fluent Bit
10	Release	CI builds+scans image -> new tag -> kubectl set image / helm upgrade -> rollout status -> rollback if bad

Apply order that avoids errors

```
kubectl apply -f namespace.yaml           # 1. namespace first
kubectl apply -f configmap.yaml -f secret.yaml # 2. things pods reference
kubectl apply -f pv-pvc.yaml              # 3. storage
kubectl apply -f mysql-statefulset.yaml    # 4. dependencies before consumers
kubectl apply -f web-deployment.yaml -f service.yaml -f ingress.yaml
kubectl apply -f hpa.yaml -f networkpolicy.yaml
kubectl get all,pvc,ingress -n shop       # 5. verify
kubectl rollout status deploy/web -n shop
```

Kubernetes retries until dependencies exist, so order is not strictly required, but this order gives clear errors and is what you should say in an interview.

Part 5 - Real Interview Questions and Answers

Answer pattern: **definition, why it exists, real example, one gotcha.**

5.1 Basics and architecture

Q1. Why Kubernetes over plain Docker?

Docker runs containers on one host. Kubernetes orchestrates across many hosts: self-healing, scaling, rolling updates, service discovery, config/secret management, storage orchestration.

Q2. Explain the architecture.

Control plane: API server, etcd, scheduler, controller-manager. Nodes: kubelet, kube-proxy, container runtime. Everything goes through the API server; etcd is the only stateful store.

Q3. What happens when you run `kubectl apply` on a Deployment?

API server authenticates/authorizes/validates and stores in etcd; Deployment controller creates a ReplicaSet and Pods; scheduler assigns nodes; kubelet pulls images and starts containers; probes decide readiness.

Q4. Pod vs container? Why not run containers directly?

A Pod wraps one or more tightly coupled containers sharing network namespace, IP and volumes. It is the scheduling unit, enabling patterns such as sidecars and init containers.

Q5. Labels vs annotations?

Labels are identifying key/values used by selectors (Services, Deployments, `kubectl -l`). Annotations are non-identifying metadata (owner, build info, tool config) and cannot be selected on.

Q6. What is a namespace and does it provide security isolation?

A virtual partition for names, quotas and RBAC. It is not a hard security boundary by itself; combine with NetworkPolicy, RBAC and PSS.

Q7. What is the difference between a declarative and imperative approach?

Imperative: `kubectl run/scale/edit` commands. Declarative: `kubectl apply -f` desired YAML kept in Git; Kubernetes reconciles. Declarative is preferred for repeatability and GitOps.

5.2 Workloads

Q8. Deployment vs StatefulSet?

Deployment: interchangeable pods, random names, shared or no storage. StatefulSet: stable names (db-0, db-1), ordered start, stable DNS via headless Service, and a dedicated PVC per pod. Use for databases, Kafka.

Q9. Deployment vs ReplicaSet?

A ReplicaSet keeps N pods running. A Deployment manages ReplicaSets to give rolling updates and rollbacks; you rarely create ReplicaSets directly.

Q10. How does a rolling update work and how do you get zero downtime?

New ReplicaSet scales up while old scales down, controlled by `maxSurge` and `maxUnavailable`. With `maxUnavailable 0` plus a correct readiness probe, a new pod must be Ready before an old one is removed.

Q11. A release is bad. What do you do?

`kubectl rollout undo deployment/web` (or `helm rollback`). Check `kubectl rollout history`. Prevention: readiness probes, canary releases, small steps.

Q12. When do you use a DaemonSet?

One pod per node: log collectors (Fluent Bit), node-exporter, CNI and storage agents.

Q13. Job vs CronJob? Important CronJob settings?

Job runs to completion; CronJob creates Jobs on a cron schedule. Set `concurrencyPolicy` (Forbid/Replace), history limits, `backoffLimit`, and `restartPolicy` Never or OnFailure.

Q14. Init container vs sidecar?

Init containers run sequentially to completion before app containers (wait for DB, migrations). Sidecars run alongside the app (log shipping, proxies, mesh).

5.3 Networking**Q15. Why do we need Services?**

Pod IPs are ephemeral. A Service provides a stable virtual IP and DNS name and load-balances across ready pods selected by labels.

Q16. ClusterIP vs NodePort vs LoadBalancer?

ClusterIP: internal only. NodePort: opens a port (30000-32767) on every node. LoadBalancer: provisions a cloud load balancer, building on NodePort.

Q17. Service vs Ingress?

Service works at L4 for one app. Ingress is L7 HTTP/HTTPS routing (host/path, TLS) for many Services behind one load balancer, and requires an ingress controller such as NGINX.

Q18. How does DNS work in Kubernetes?

CoreDNS resolves service.namespace.svc.cluster.local. Pods in the same namespace can use just the service name.

Q19. What is a headless Service?

clusterIP: None. No virtual IP; DNS returns pod IPs directly. Used by StatefulSets for stable per-pod DNS.

Q20. How do you restrict traffic between pods?

NetworkPolicy: default-deny then allow specific podSelector/namespaceSelector and ports. Needs a CNI that enforces it (Calico, Cilium).

5.4 Storage and configuration**Q21. Explain PV, PVC, StorageClass.**

PV is the actual storage (cluster-scoped). PVC is a namespaced request for storage. StorageClass defines how to provision dynamically (EBS gp3, etc.), so a PVC automatically creates the PV.

Q22. Reclaim policies and access modes?

Retain keeps data after PVC deletion; Delete removes the disk. RWO: one node, ROX: many read-only, RWX: many read-write (NFS/EFS). Block disks like EBS are RWO.

Q23. ConfigMap vs Secret? Are Secrets secure?

Both inject config via env or files. Secrets are only base64-encoded by default and stored plain in etcd. Secure them with etcd encryption at rest, RBAC, and external managers (Vault, AWS Secrets Manager).

Q24. If you update a ConfigMap, do pods see it?

Mounted files update after a short delay (not with subPath). Env vars do not change until pods restart (rollout restart or a checksum annotation in Helm).

Q25. What happens to StatefulSet data when you delete a pod? Delete the StatefulSet?

Pod deletion: it returns with the same name and PVC. Deleting the StatefulSet leaves PVCs behind by default, which protects data.

5.5 Scaling and scheduling**Q26. HPA vs VPA vs Cluster Autoscaler?**

HPA changes pod count by metrics. VPA changes pod CPU/memory requests. Cluster Autoscaler adds/removes nodes when pods are Pending or nodes are idle. HPA + CA is the common production pair.

Q27. HPA is not scaling. Why?

Metrics Server missing, no CPU requests set, target pod not running, max replicas reached, or the metric is below target. Check `kubectl describe hpa` and `kubectl top pods`.

Q28. Requests vs limits? What happens when exceeded?

Requests: reserved amount used for scheduling. Limits: ceiling. Exceed CPU limit: throttled. Exceed memory limit: OOMKilled (exit 137).

Q29. Taints vs node affinity?

Taints on nodes repel pods unless they have a matching toleration (dedicated GPU nodes). Affinity on pods attracts them to nodes. Tolerating a taint does not force scheduling there; combine with affinity/nodeSelector.

Q30. Explain liveness, readiness and startup probes.

Liveness failing restarts the container. Readiness failing removes the pod from Service endpoints. Startup delays the other probes for slow-starting apps. A wrong liveness probe can cause restart loops.

Q31. ResourceQuota vs LimitRange?

Quota caps total consumption per namespace. LimitRange sets per-container defaults and min/max.

5.6 Administration, security, advanced

Q32. Explain RBAC.

Role/ClusterRole define verbs on resources; RoleBinding/ClusterRoleBinding attach them to users, groups or ServiceAccounts. Test with `kubectl auth can-i`. Follow least privilege.

Q33. Role vs ClusterRole?

Role is namespaced. ClusterRole is cluster-wide or for non-namespaced resources (nodes, PVs) and can be bound per namespace with a RoleBinding.

Q34. How do you upgrade a cluster?

Back up etcd, read release notes, upgrade control plane, then each node one at a time: drain, upgrade kubelet, uncordon. One minor version at a time. Managed services automate this.

Q35. What are CRDs and Operators?

CRD adds a new API object type. An Operator is a CRD plus a controller encoding operational knowledge (backup, failover) using the reconcile loop. Example: a Postgres operator.

Q36. Helm: what and why?

Package manager with templated charts and values. Gives versioned releases, upgrade and rollback, and per-environment configuration. `helm template` previews the rendered YAML.

Q37. What is a service mesh and when not to use it?

Sidecar proxies (Istio, Linkerd) provide mTLS, retries, traffic splitting, observability. Skip it for few services because of complexity and resource overhead.

Q38. What are Pod Security Standards?

Built-in levels (privileged, baseline, restricted) enforced per namespace by labels; replaced PodSecurityPolicy. Restricted requires non-root, no privilege escalation, dropped capabilities.

Q39. How do you secure a cluster end to end?

Scan and sign images, run non-root with securityContext, PSS, default-deny NetworkPolicies, least-privilege RBAC, encrypted Secrets or external vault, audit logs, patch regularly, restrict API server access.

Q40. How do you handle secrets in GitOps?

Never commit plain secrets. Use Sealed Secrets, SOPS, or External Secrets Operator pulling from a secret manager.

5.7 Cloud-native and operations

Q41. Why managed Kubernetes (EKS/AKS/GKE)?

The provider runs and patches the highly available control plane and etcd; you manage nodes and workloads. It integrates with cloud load balancers, disks and IAM.

Q42. How do Spot nodes fit in production?

Use for stateless, fault-tolerant workloads: tainted Spot node group, tolerations, multiple instance types, 2+ replicas, PodDisruptionBudget and graceful SIGTERM handling. Keep databases on on-demand nodes.

Q43. What does a PodDisruptionBudget do?

Limits voluntary disruptions (drains, upgrades, autoscaler) so a minimum number of pods stays available.

Q44. Cluster Autoscaler vs Karpenter?

Cluster Autoscaler scales pre-defined node groups. Karpenter provisions right-sized nodes directly from pending pods, usually faster and cheaper.

5.8 Troubleshooting and scenario questions

Symptom	Likely cause	How you investigate / fix
Pending	Insufficient CPU/memory, unbound PVC, taint/affinity mismatch	describe pod -> FailedScheduling event; lower requests, add nodes, fix tolerations, check PVC
ImagePullBackOff	Wrong tag/name, private registry without secret	describe pod; fix image; create docker-registry secret + imagePullSecrets
CrashLoopBackOff	App exits: bad config, missing env, dependency down	logs --previous; check ConfigMap/Secret, command, probes
OOMKilled (137)	Memory above limit or leak	describe pod (Last State), kubectl top; raise limit or fix leak
Running but 0/1 Ready	Readiness probe failing	check probe path/port, app logs
CreateContainerConfigError	Missing ConfigMap/Secret/key, wrong namespace	describe pod; create object or fix reference
Service unreachable	Selector mismatch, wrong targetPort, no Ready pods, NetworkPolicy	get endpoints; port-forward to the pod, then the Service, then Ingress
Evicted	Node pressure (disk/memory)	describe node; set requests/limits, add capacity
Forbidden	RBAC	kubectl auth can-i; fix Role/Binding

Scenario questions

Q45. Users report the site is down. Walk me through your debugging.

1) kubectl get pods/deploy: are pods Running and Ready? 2) describe/logs --previous for failing pods. 3) get endpoints for the Service. 4) Check Ingress and controller logs. 5) Check recent changes (rollout history, events). 6) Roll back if a release caused it, then do root-cause analysis.

Q46. Pod is CrashLoopBackOff but logs are empty.

Use logs --previous; describe for exit code and events (OOMKilled 137, command not found 127). If the image has no shell use kubectl debug with an ephemeral container. Check liveness probe killing a slow app.

Q47. Your app must connect to the DB only on startup order. How?

Do not rely on ordering. Use an init container waiting for the DB port and make the app retry with backoff; readiness probe checks the DB connection.

Q48. How do you do a canary or blue/green release?

Blue/green: two Deployments, switch the Service selector. Canary: small replica share of the new version, or weighted routing via Ingress annotations/Istio/Argo Rollouts; promote on healthy metrics.

Q49. Costs are too high. What do you check?

Over-sized requests (VPA recommendations, kubectl top), idle nodes (autoscaler scale-down), Spot for stateless work, right-sized instances, namespace quotas, unused PVCs/load balancers.

Q50. Node goes NotReady. What happens and what do you do?

After the eviction timeout, controllers recreate its pods elsewhere (StatefulSets may need care). Investigate with describe node (conditions), kubelet logs, disk/memory pressure; cordon/drain and replace the node if needed.

Part 6 - Cheat Sheet and Revision Plan

```
# inspect
kubectl get pods,svc,deploy -A -o wide
kubectl get <kind> <name> -o yaml
kubectl api-resources
# change
kubectl apply -f file.yaml
kubectl diff -f file.yaml
kubectl scale deploy web --replicas=5
kubectl rollout status|history|undo|restart deploy/web
kubectl label|annotate <kind> <name> k=v
# debug
kubectl logs <pod> -c <ctr> -f --previous
kubectl port-forward svc/web 8080:80
kubectl top nodes|pods
# nodes
kubectl cordon|drain|uncordon <node>
# generate YAML fast
kubectl create deploy web --image=nginx --dry-run=client -o yaml > web.yaml
kubectl expose deploy web --port=80 --dry-run=client -o yaml

kubectl describe <kind> <name>
kubectl explain <kind>.spec --recursive
kubectl get events --sort-by=.lastTimestamp
kubectl delete -f file.yaml
kubectl edit deploy web
kubectl set image deploy/web web=img:tag
kubectl patch <kind> <name> -p '{...}'
kubectl exec -it <pod> -- sh
kubectl debug -it <pod> --image=busybox
kubectl auth can-i <verb> <res> --as=<user>
kubectl taint node <n> key=val:NoSchedule
```

7-day revision plan

Day	Topic (repo folder)	Do this
1	Setup, core concepts (00, 01)	Create kind cluster; pods, labels, init/sidecar
2	Workloads (02, my-labs/nginx)	Deploy, rolling update, rollback; Job + CronJob
3	Networking (03)	Service types, Ingress, NetworkPolicy; break a selector on purpose
4	Storage + config (04, my-labs/mysql)	PVC, StatefulSet data survives pod deletion; Secret
5	Scaling + scheduling (05, my-labs/apache)	HPA load test, taints/tolerations, quotas
6	Admin, security, monitoring (06, 07, 09)	RBAC can-i tests, PSS label, Prometheus via Helm
7	Cloud, troubleshooting, mock interview (10, 11)	Fix every pod in broken-pods.yaml; answer Part 5 aloud

Last-minute interview tips

1. Always explain **problem it solves** before the definition.
2. Tie answers to a real example (shop, MySQL, nightly backup, Black Friday traffic).
3. For troubleshooting, state your **method**: get, describe, logs, events, then network layers in order.
4. Mention trade-offs and gotchas (Secrets are base64, HPA needs requests, Spot is for stateless).
5. If unsure of a field, say you would check `kubectl explain` - that is exactly what practitioners do.